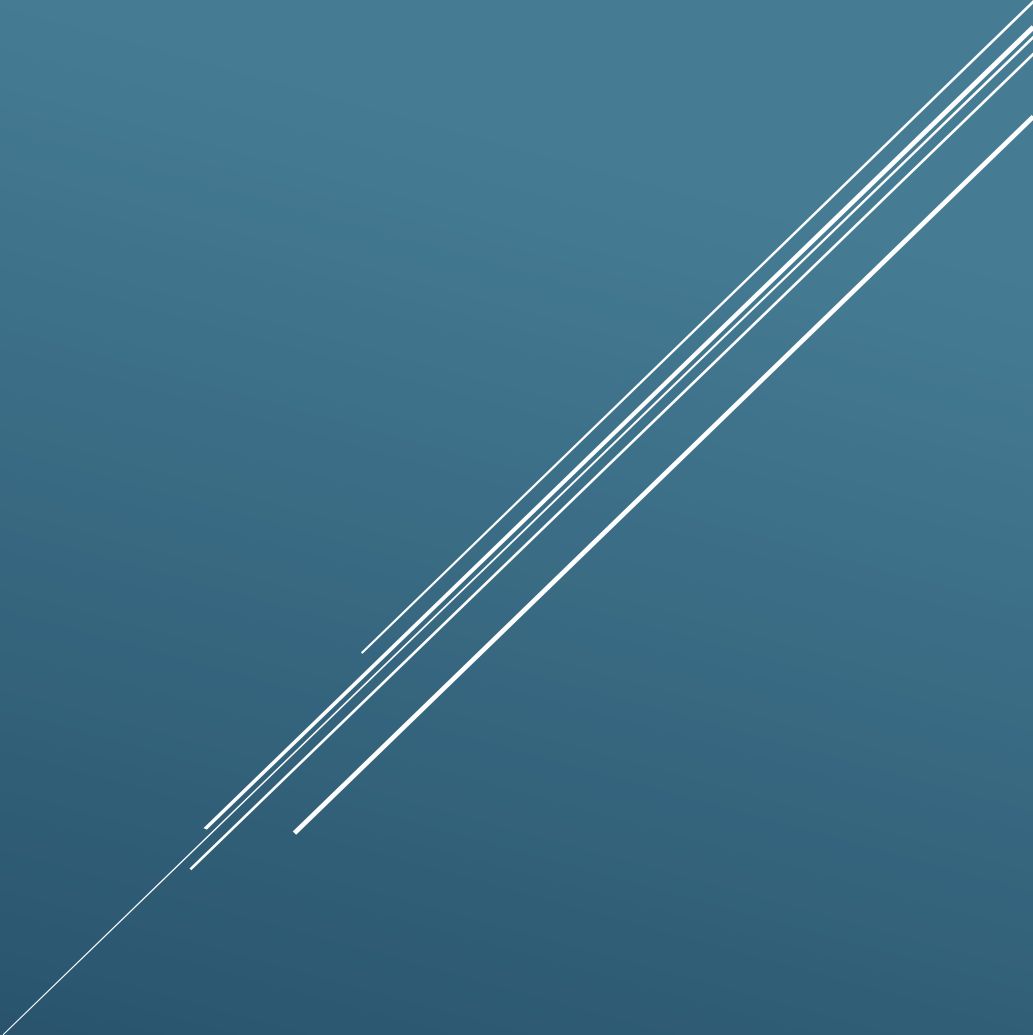




SA.5 — E2.2: PLAN Y REPORTE DE PRUEBAS

Josue Arreola Salinas IDGS-16

Índice

1. Introducción.....	2
2. Objetivo	2
3. Plan de pruebas	2
3.1 Consideraciones del entorno de pruebas.....	2
4. Casos de prueba	3
Caso de prueba 1 – Consumo de API (P2.5)	3
Caso de prueba 2 – Manejo de error de red (P2.5).....	5
Caso de prueba 3 – Comunicación Wear OS → Teléfono (P2.6)	6
Caso de prueba 4 – Navegación mediante D-pad (PWA)	8
Caso de prueba 5 – Funcionamiento Offline	9
Caso de prueba 6 – Sincronización Teléfono → Smart TV	11
Caso de prueba 7 – Visualización de datos reales	12
Caso de prueba 8 – Actualización en tiempo real	13
Caso de prueba 9 – Estabilidad de la aplicación.....	14
Caso de prueba 10 – Funcionamiento simultáneo del ecosistema.....	15
5. Evidencias	16
6. Conclusiones	17
Justificación del uso de Inteligencia Artificial	18

1. Introducción

Con el propósito de verificar el correcto funcionamiento del ecosistema de dispositivos inteligentes desarrollado para el sistema de gestión de citas de la barbería, se realizaron diversas pruebas funcionales sobre cada uno de los componentes que integran la solución. El ecosistema está conformado por una aplicación móvil desarrollada en Flutter, una aplicación para dispositivos Wear OS, una Progressive Web App (PWA) para Smart TV y un backend desarrollado con NestJS conectado a una base de datos PostgreSQL mediante Neon.

Las pruebas permitieron comprobar la comunicación entre los diferentes dispositivos, el consumo de datos reales desde la API, la sincronización de información en tiempo real, el funcionamiento de la Progressive Web App en modo offline y la navegación mediante control remoto virtual (D-pad), garantizando que todos los componentes operaran correctamente de manera integrada.

2. Objetivo

Documentar las pruebas realizadas sobre el ecosistema de dispositivos inteligentes con el fin de verificar el correcto funcionamiento de la aplicación móvil, la aplicación Wear OS, la Progressive Web App para Smart TV y la comunicación entre todos los componentes del sistema, cumpliendo con los criterios establecidos en la Evaluación 2 de la asignatura Desarrollo para Dispositivos Inteligentes.

3. Plan de pruebas

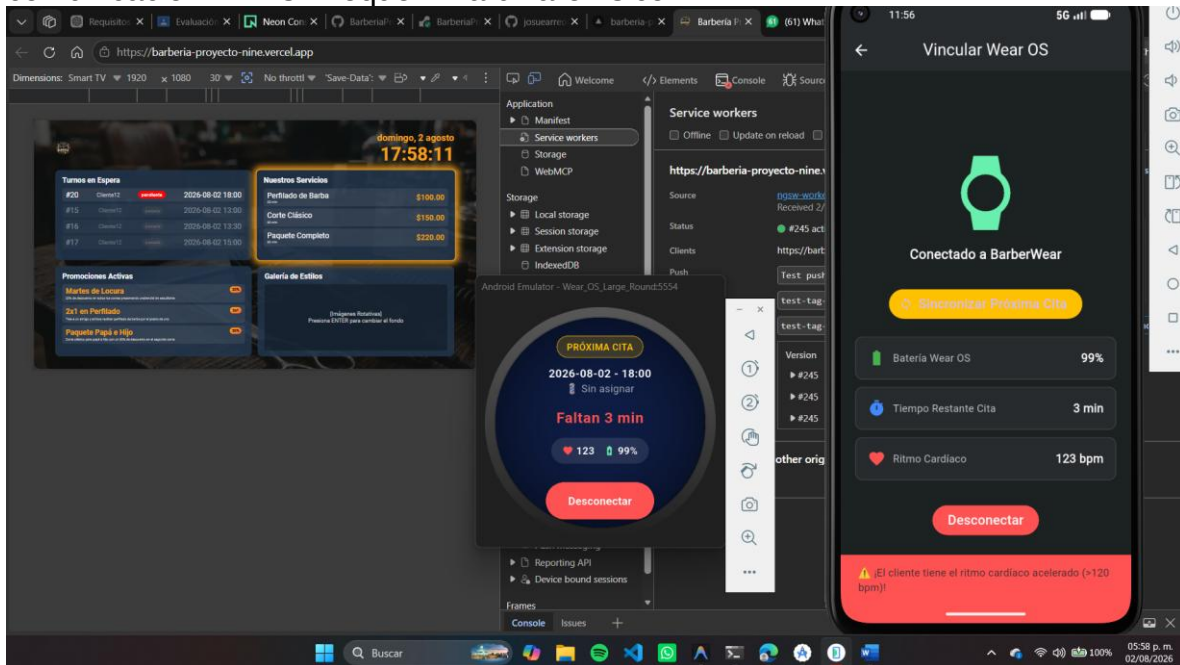
3.1 Consideraciones del entorno de pruebas

Las pruebas fueron realizadas utilizando los emuladores oficiales de Android Studio para la aplicación móvil y la aplicación Wear OS, mientras que la Progressive Web App fue ejecutada utilizando Google Chrome con la emulación de un dispositivo Smart TV mediante Chrome DevTools.

Es importante mencionar que los emuladores de Android Studio no cuentan con soporte para emular el hardware Bluetooth de un dispositivo físico, por lo que no es posible establecer una comunicación Bluetooth Low Energy (BLE) real entre ambos emuladores.

Sin embargo, el proyecto conserva implementada toda la comunicación BLE necesaria para funcionar en dispositivos físicos, incluyendo el escaneo mediante `flutter_blue_plus`, la exposición de servicios y características GATT mediante `ble_peripheral`, el uso de UUID compartidos y la comunicación mediante la propiedad `NOTIFY`.

Para poder validar el flujo completo durante las pruebas de integración, se implementó una capa de comunicación alternativa mediante **Socket.IO**, permitiendo que el emulador Wear OS enviara las métricas simuladas al backend y éste las retransmitiera inmediatamente al emulador del teléfono. Esta estrategia permitió verificar la generación, transmisión, recepción y visualización de la información en tiempo real, reproduciendo el comportamiento esperado de la comunicación BLE sin requerir hardware físico.



4. Casos de prueba

Caso de prueba 1 – Consumo de API (P2.5)

Objetivo

Verificar que la aplicación móvil pueda autenticarse correctamente y obtener información real desde la API del sistema.

Procedimiento

1. Iniciar la aplicación móvil.
2. Introducir credenciales válidas.
3. Presionar el botón de iniciar sesión.
4. Esperar la respuesta del servidor.

Resultado esperado

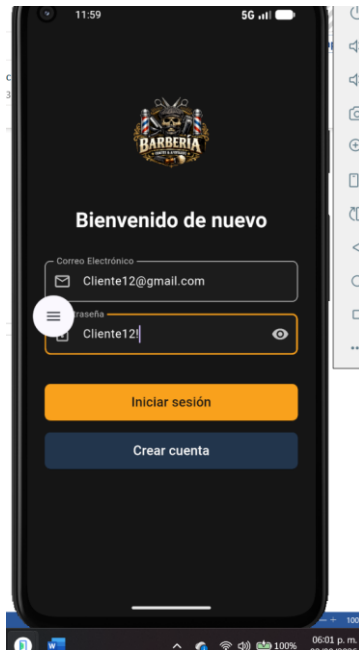
La aplicación obtiene un token JWT válido, inicia sesión correctamente y muestra la información del usuario y sus citas obtenidas desde la API.

Resultado obtenido

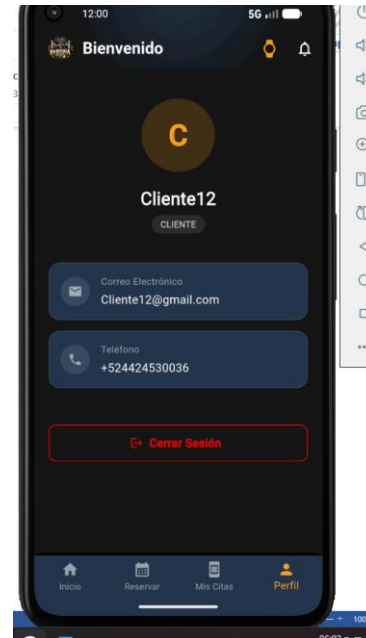
La autenticación se realizó correctamente, el servidor respondió con un token válido y la aplicación cargó la información correspondiente sin presentar errores.

Evidencia:

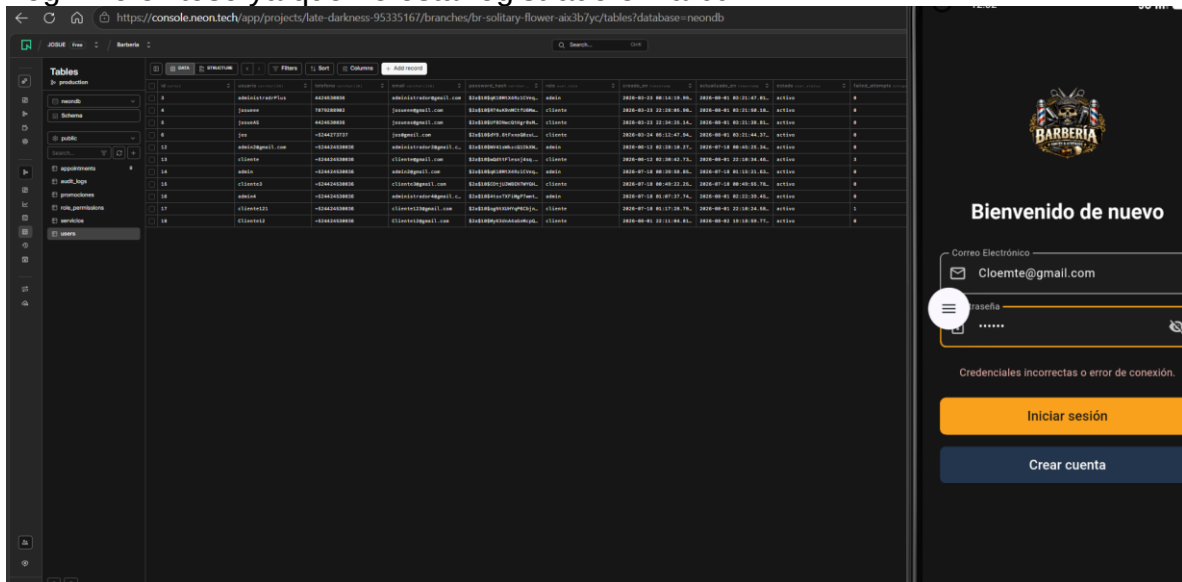
Login valido:



Sesion iniciada con ese perfil:



Login no exitoso ya que no está registrado en la bd



Caso de prueba 2 – Manejo de error de red (P2.5)

Objetivo

Verificar el comportamiento de la aplicación cuando no existe conexión con el servidor.

Procedimiento

1. Desactivar la conexión a Internet del emulador.
2. Intentar iniciar sesión.
3. Observar el comportamiento de la aplicación.

Resultado esperado

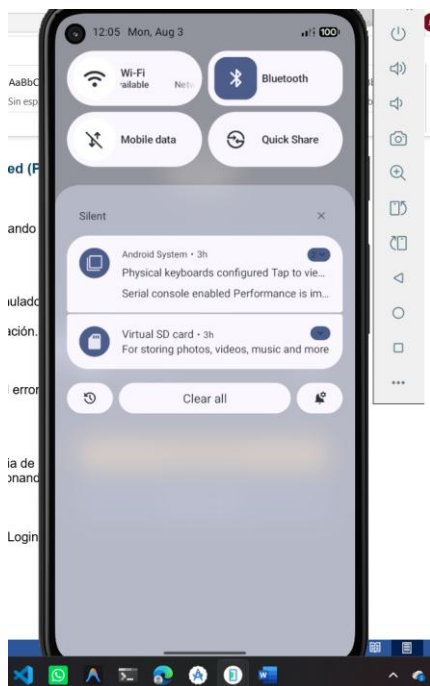
La aplicación muestra un mensaje indicando el error de conexión y permanece estable sin cerrarse inesperadamente.

Resultado obtenido

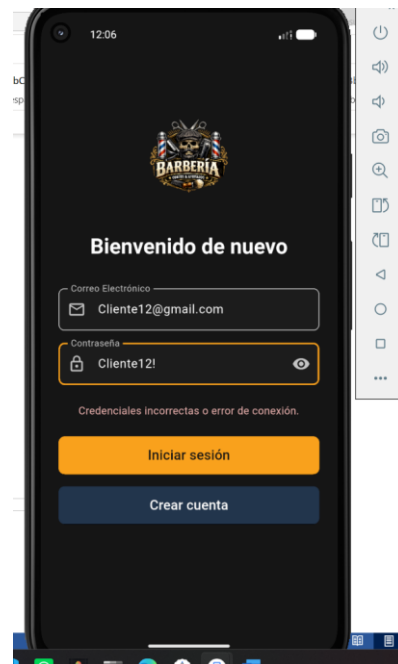
La aplicación detectó correctamente la ausencia de conexión, mostró el mensaje correspondiente al usuario y permaneció funcionando normalmente.

Evidencia:

Emulador con internet desactivado:



Login erróneo con credenciales validas:



Caso de prueba 3 – Comunicación Wear OS → Teléfono (P2.6)

Objetivo

Verificar la transmisión de información generada por el wearable hacia la aplicación móvil.

Procedimiento

1. Ejecutar la aplicación Wear OS.
2. Iniciar la simulación de sensores.
3. Verificar que las métricas sean recibidas por la aplicación móvil.

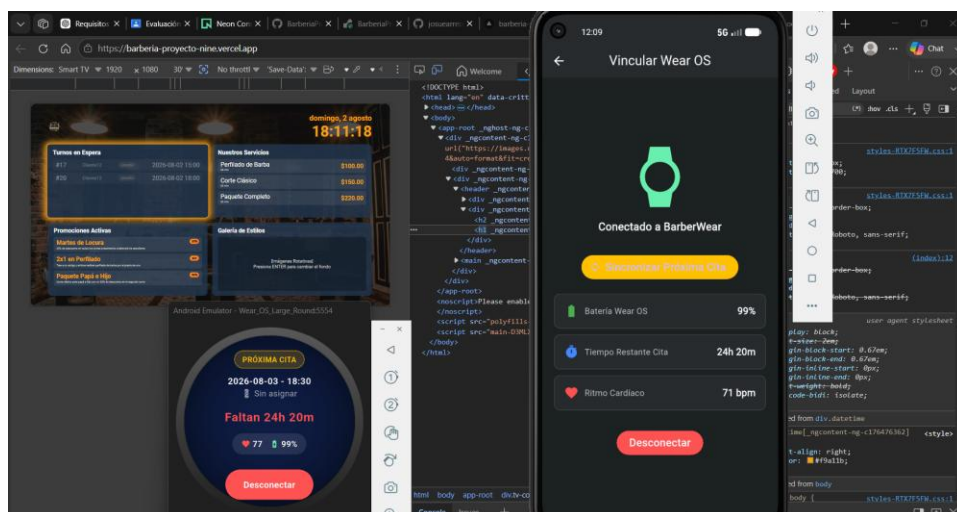
Resultado esperado

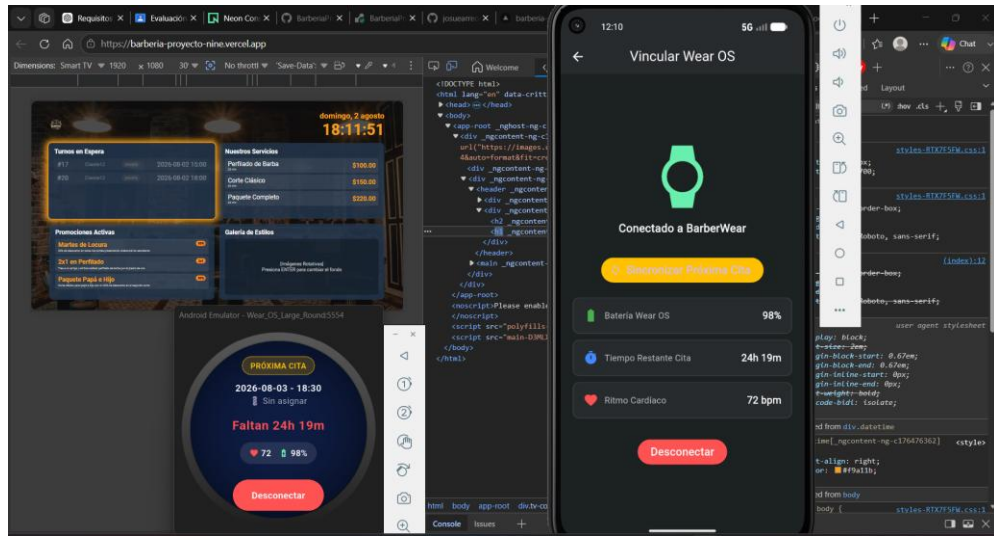
Las métricas del wearable son recibidas y actualizadas en tiempo real dentro de la aplicación móvil.

Resultado obtenido

Las métricas de batería, ritmo cardíaco y tiempo restante fueron transmitidas correctamente mediante la capa de simulación basada en Socket.IO, reproduciendo el comportamiento esperado de la comunicación BLE implementada en el proyecto.

Evidencia:





Caso de prueba 4 – Navegación mediante D-pad (PWA)

Objetivo

Verificar la navegación de la interfaz utilizando las teclas direccionales.

Procedimiento

1. Abrir la PWA para Smart TV.
2. Utilizar las teclas Arrow Up, Arrow Down, Arrow Left y Arrow Right.
3. Presionar la tecla Enter.

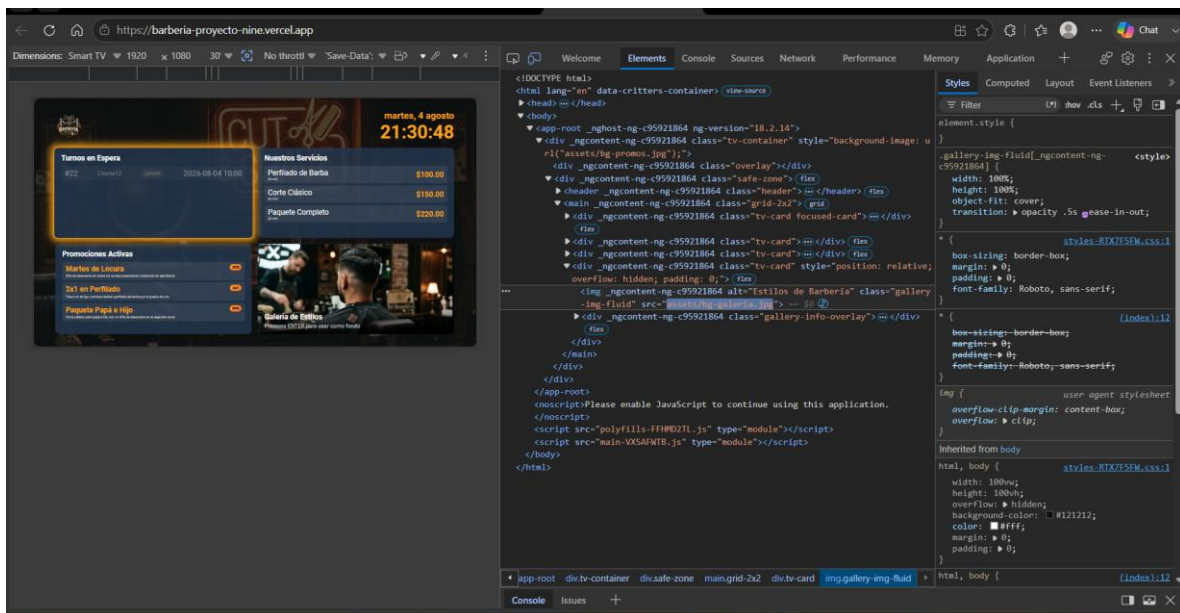
Resultado esperado

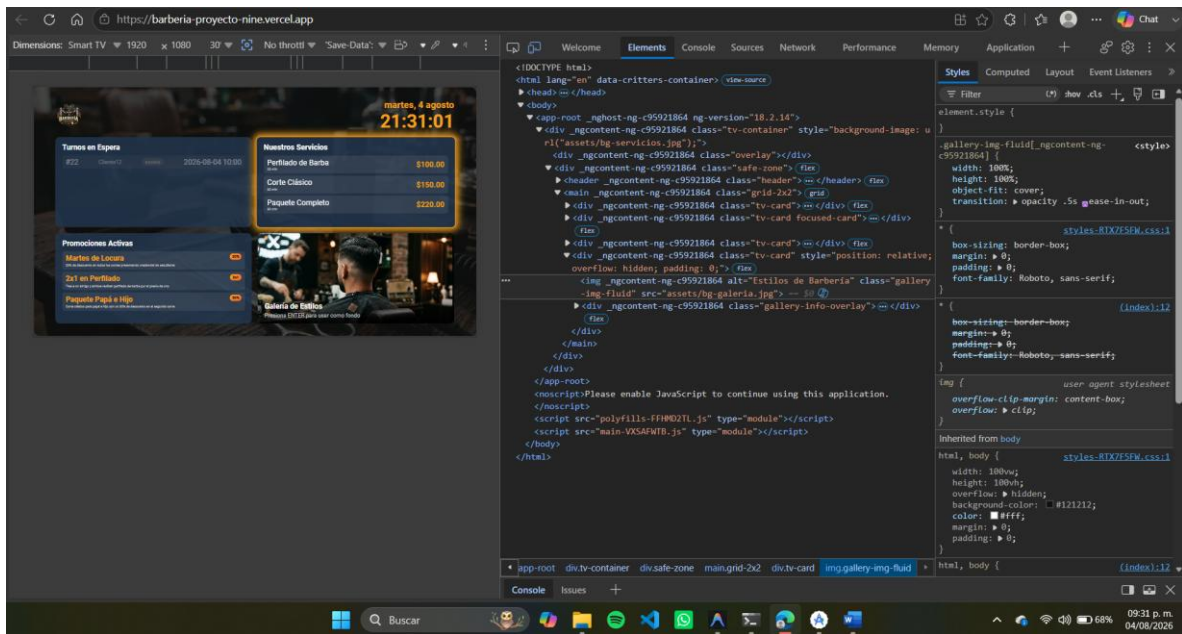
El foco cambia correctamente entre las tarjetas y la tecla Enter actualiza el recurso multimedia correspondiente.

Resultado obtenido

La navegación mediante D-pad funcionó correctamente, respetando los límites del grid y permitiendo seleccionar cada elemento de la interfaz.

Evidencia:





Caso de prueba 5 – Funcionamiento Offline

Objetivo

Verificar el funcionamiento del Service Worker.

Procedimiento

1. Abrir Chrome DevTools.
2. Activar el modo Offline.
3. Recargar la aplicación.

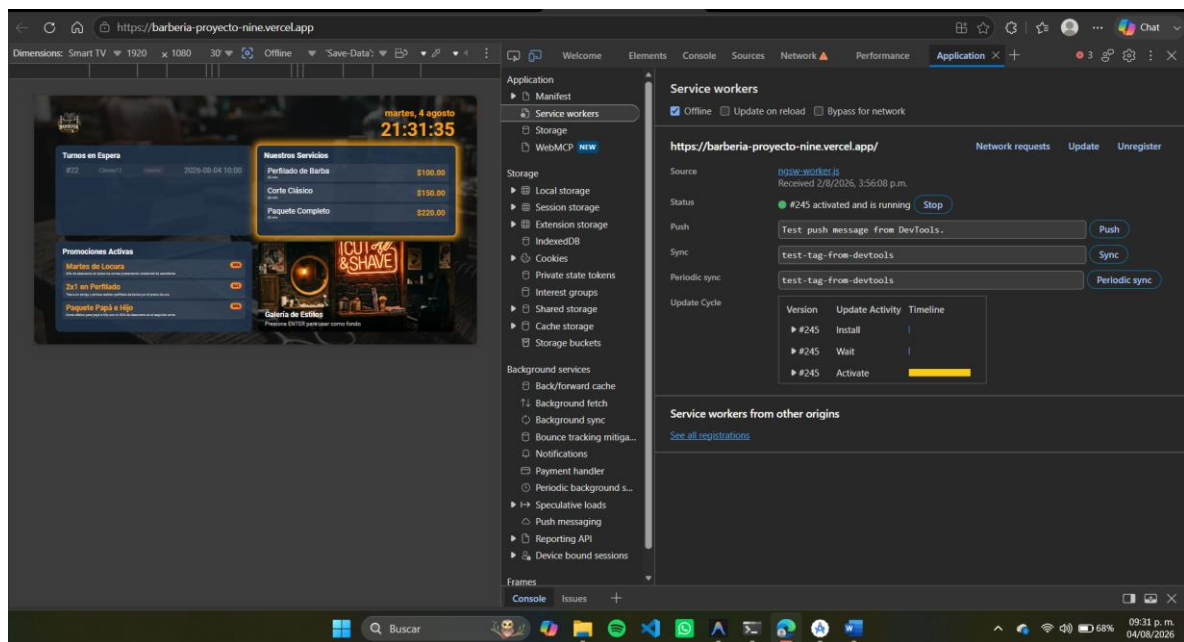
Resultado esperado

La aplicación continúa mostrando la estructura principal utilizando los recursos almacenados en caché.

Resultado obtenido

El Service Worker cargó correctamente los recursos almacenados, permitiendo que la aplicación continuara funcionando sin conexión.

Evidencia:



Caso de prueba 6 – Sincronización Teléfono → Smart TV

Objetivo

Verificar que los cambios realizados desde la aplicación móvil sean reflejados en la Smart TV.

Procedimiento

1. Crear o modificar una cita desde el teléfono.
2. Esperar la actualización de la Smart TV.

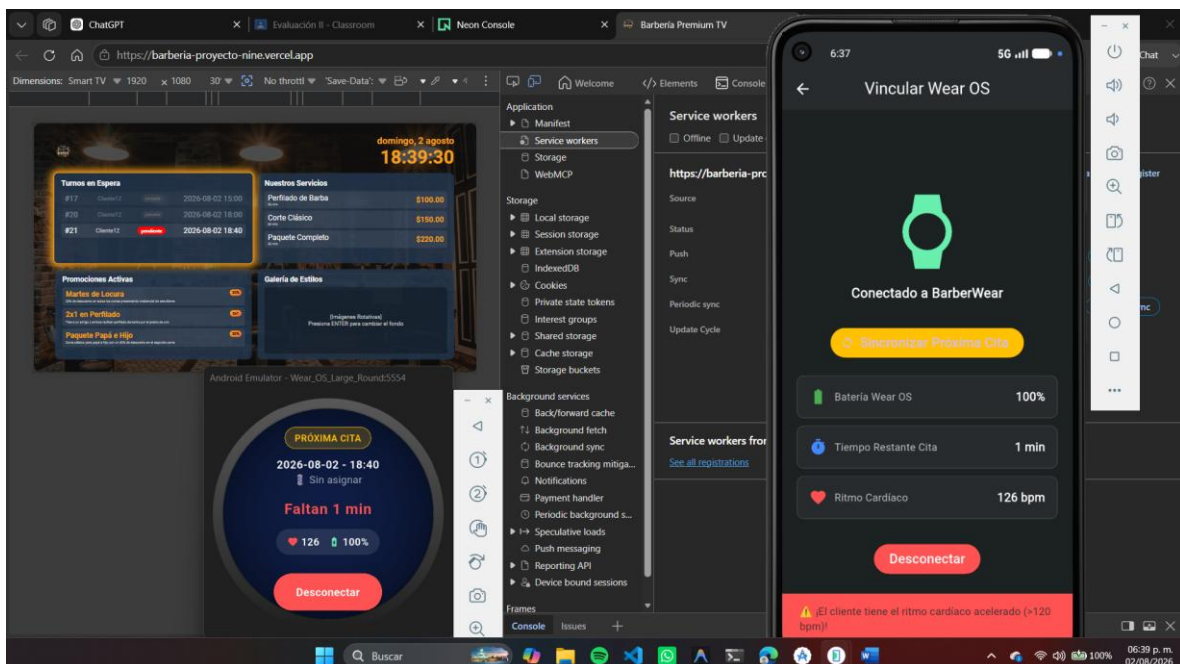
Resultado esperado

La información aparece actualizada en la Smart TV en menos de dos segundos.

Resultado obtenido

La sincronización se realizó correctamente mediante WebSockets, reflejando la información actualizada casi de forma inmediata.

Evidencia:



Caso de prueba 7 – Visualización de datos reales

Objetivo

Verificar que la Smart TV muestre información obtenida directamente desde la API.

Procedimiento

1. Abrir la PWA.
2. Consultar los apartados de citas, servicios y promociones.

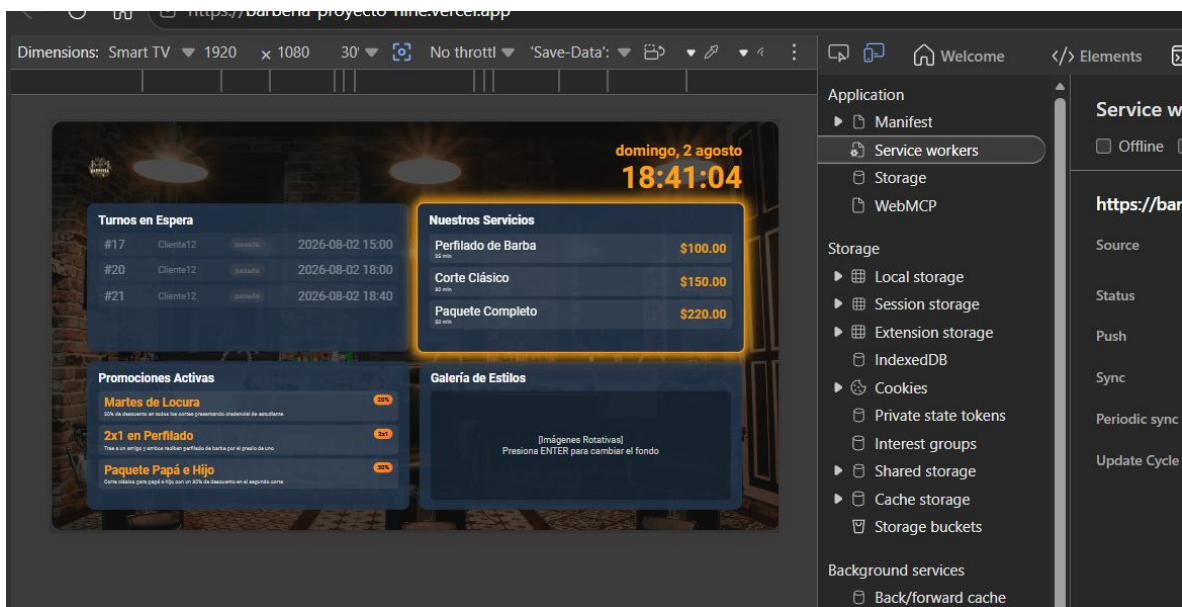
Resultado esperado

La información mostrada corresponde a los datos almacenados en la base de datos.

Resultado obtenido

La PWA mostró correctamente la información obtenida desde la API, incluyendo citas, promociones y servicios.

Evidencia:



id	nombre	descripcion	descuento	fecha_inicio	fecha_fin	activo	creado_en	actualizado_en
1	Martes de Locura	20% de descuento en to...	20%	2020-08-01	2020-12-31	TRUE	2020-08-02 20:24:51.39...	2020-08-02 20:24:51.39...
2	2x1 en Perfilado	Tras a un amigo y amba...	2x1	2020-08-01	2020-09-30	TRUE	2020-08-02 20:24:51.39...	2020-08-02 20:24:51.39...
3	Paquete Popi e Hijo	Corte clásico para pap...	30%	2020-08-01	2020-10-31	TRUE	2020-08-02 20:24:51.39...	2020-08-02 20:24:51.39...

id	nombre	descripcion	precio	duracion_minutos	activo	creado_en	actualizado_en
1	Corte Clásico	Corte de cabello tradi...	150.00	30	TRUE	2020-08-02 20:24:11.31...	2020-08-02 20:24:11.31...
2	Perfilado de Barba	Perfilado y diseño de ...	100.00	25	TRUE	2020-08-02 20:24:11.31...	2020-08-02 20:24:11.31...
3	Paquete Completo	Corte de cabello + per...	220.00	50	TRUE	2020-08-02 20:24:11.31...	2020-08-02 20:24:11.31...

Caso de prueba 8 – Actualización en tiempo real

Objetivo

Verificar la actualización automática de información recibida mediante Socket.IO.

Procedimiento

1. Registrar una nueva cita.
2. Observar la actualización en la Smart TV.

Resultado esperado

La nueva cita aparece automáticamente sin necesidad de recargar la página.

Resultado obtenido

La información se actualizó correctamente mediante eventos emitidos por el servidor.

Caso de prueba 9 – Estabilidad de la aplicación

Objetivo

Verificar el comportamiento del sistema durante el uso continuo.

Procedimiento

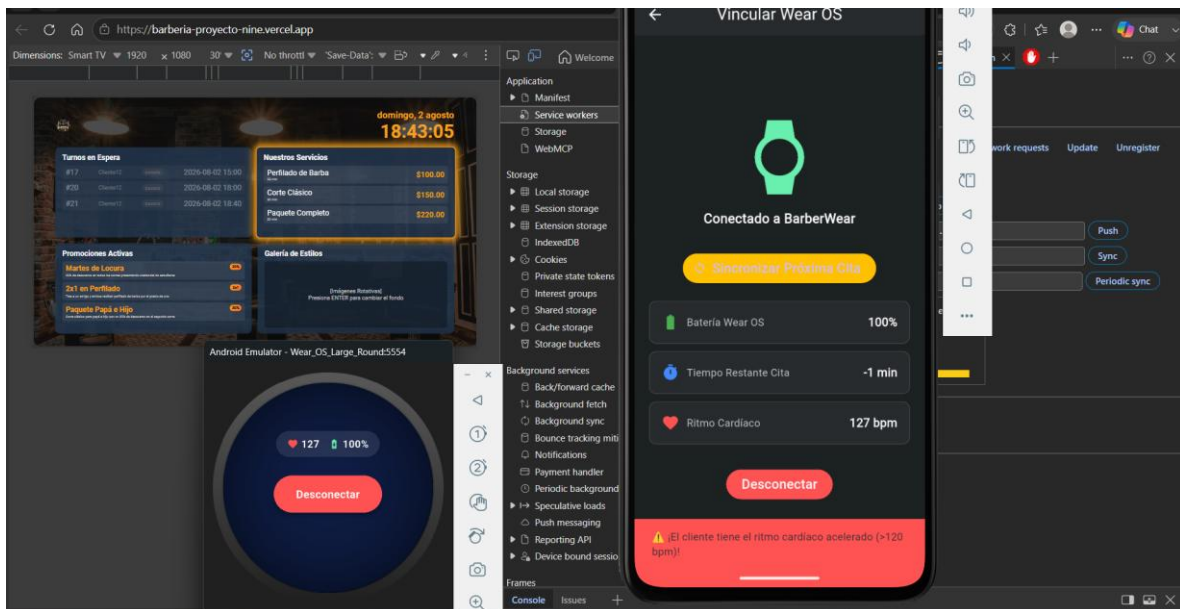
1. Mantener los tres dispositivos ejecutándose durante cinco minutos.
2. Navegar entre las diferentes funciones del sistema.

Resultado esperado

Ninguna aplicación presenta cierres inesperados o pérdida de sincronización.

Resultado obtenido

El ecosistema permaneció estable durante toda la demostración.



Caso de prueba 10 – Funcionamiento simultáneo del ecosistema

Objetivo

Verificar la operación conjunta de los tres dispositivos.

Procedimiento

1. Ejecutar la aplicación móvil.
2. Ejecutar la aplicación Wear OS.
3. Ejecutar la Smart TV.
4. Comprobar la comunicación entre todos los componentes.

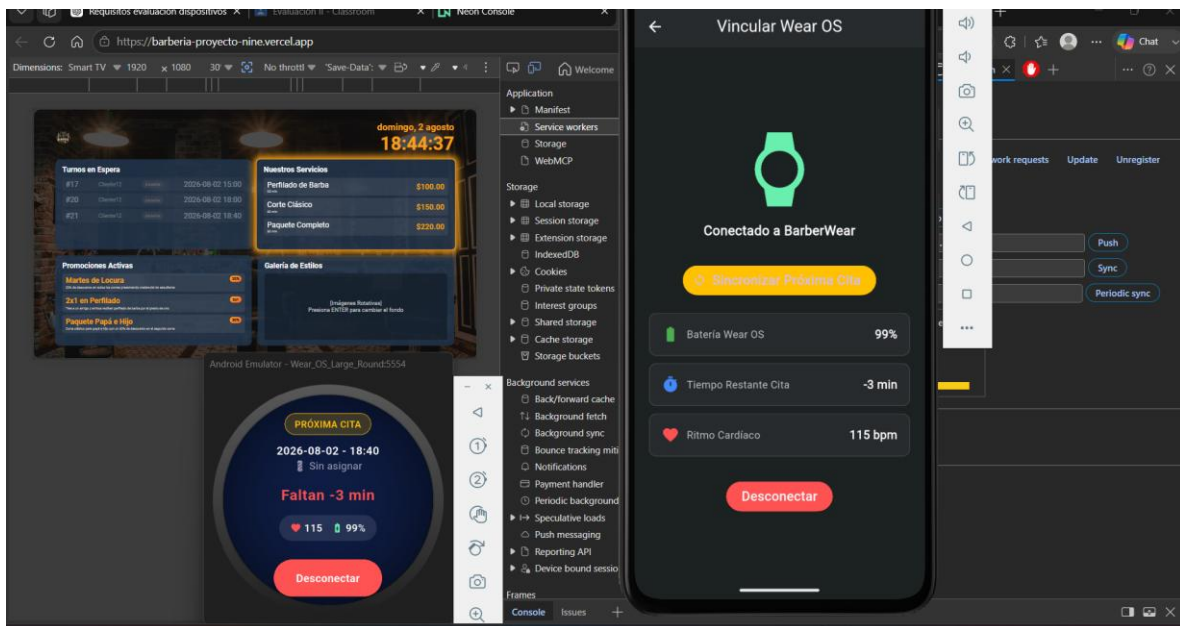
Resultado esperado

Los tres dispositivos funcionan simultáneamente manteniendo la sincronización de la información.

Resultado obtenido

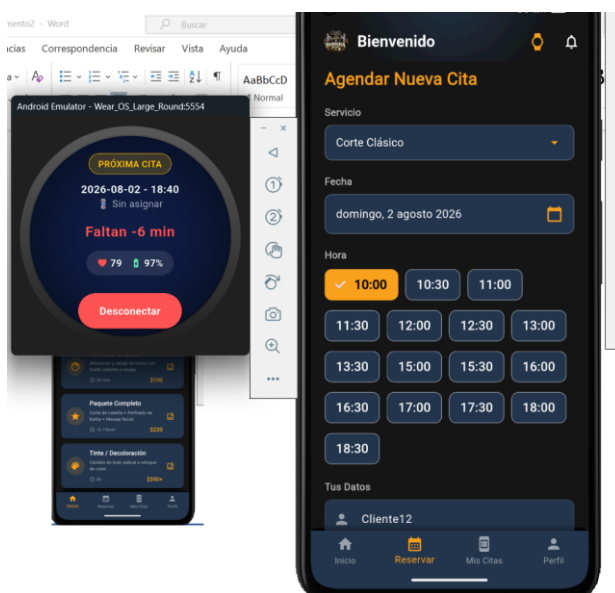
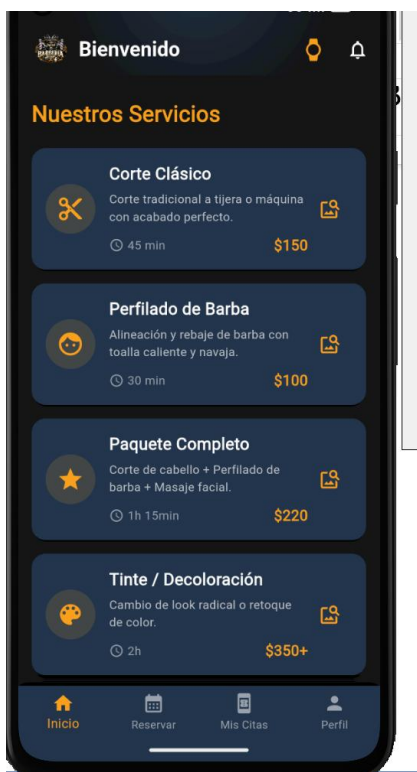
El ecosistema operó correctamente durante toda la demostración, manteniendo la comunicación entre los tres dispositivos.

Evidencia:

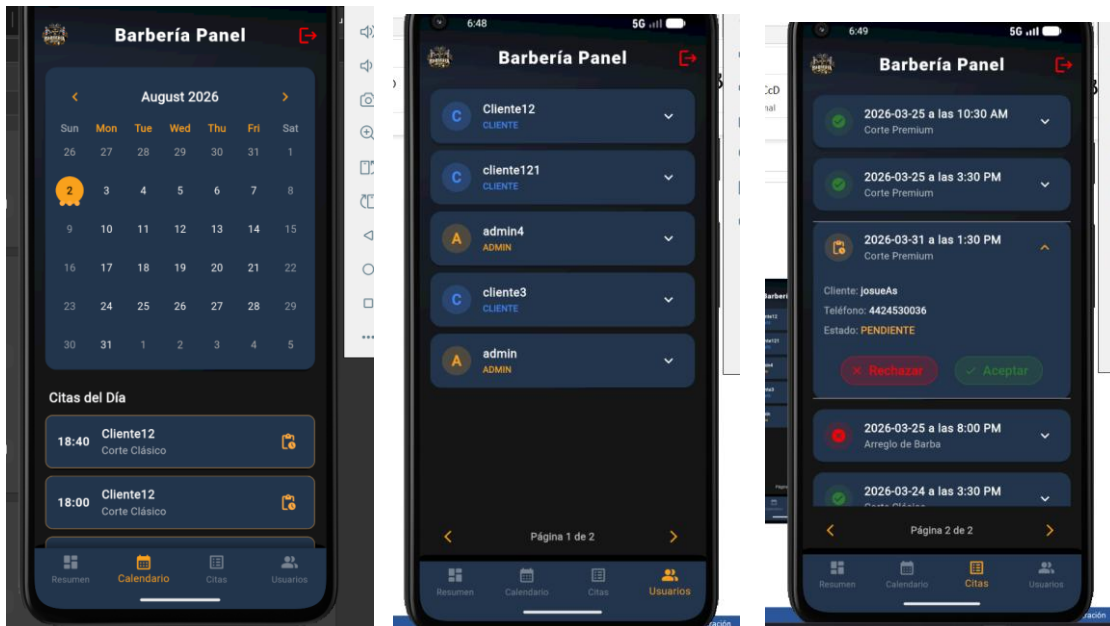


5. Evidencias

En esta sección se presentan las capturas obtenidas durante la ejecución de las pruebas realizadas al ecosistema de dispositivos inteligentes. Las evidencias muestran el funcionamiento de la aplicación móvil, la aplicación Wear OS, la Progressive Web App para Smart TV y la comunicación entre los diferentes componentes del sistema.



Vista de administrador



6. Conclusiones

Las pruebas realizadas permitieron verificar el correcto funcionamiento del ecosistema desarrollado, comprobando la integración entre la aplicación móvil, la aplicación Wear OS, la Progressive Web App para Smart TV y el backend del sistema. Los resultados obtenidos demostraron que la solución es capaz de consumir datos reales desde la API, sincronizar información entre dispositivos en tiempo real, mantener el funcionamiento de la PWA mediante el Service Worker y ofrecer una navegación adecuada utilizando el esquema 10-foot mediante D-pad.

Asimismo, se comprobó que, aunque los emuladores oficiales de Android Studio no permiten establecer una comunicación Bluetooth Low Energy real, la implementación BLE permanece integrada dentro del proyecto y fue posible validar el flujo completo de intercambio de información utilizando una capa de simulación basada en Socket.IO. Esta estrategia permitió reproducir el comportamiento esperado de la comunicación entre el wearable y la aplicación móvil sin requerir dispositivos físicos, cumpliendo con los objetivos funcionales del proyecto y con los criterios establecidos en la rúbrica de evaluación.

Justificación del uso de Inteligencia Artificial

Durante la elaboración del software y la documentación del proyecto se utilizó una herramienta de Inteligencia Artificial (ChatGPT, OpenAI) como apoyo en la detección y solución de problemas relacionados con el código, análisis de errores, consulta de posibles soluciones técnicas y mejora de la redacción del contenido.

La Inteligencia Artificial fue utilizada como una herramienta complementaria durante el proceso de desarrollo, pero no realizó la programación del sistema ni sustituyó el análisis, la lógica de implementación, las pruebas o la validación del funcionamiento del software. La arquitectura, desarrollo, decisiones técnicas y comprobaciones del sistema fueron realizadas por el autor del proyecto.

En la elaboración del documento, la información y contenido técnico fueron redactados previamente por el autor con base en el trabajo realizado, evidencias obtenidas y pruebas ejecutadas. Posteriormente, la Inteligencia Artificial fue utilizada únicamente para apoyar en la corrección de redacción, ortografía, estructura y claridad del texto, manteniendo la responsabilidad del contenido final en el autor.

Nombre y Firma: Josué Arreola Salinas

Fecha: 07 de agosto del 2026

